CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

Presentation Script and Viva Preparation

Team 20

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Deependra Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

The brief states that **any team member may be asked to explain any formula or result**. Section 3 of this document is the preparation for that, and is the part worth rehearsing.

---

0. The built deck

The slides below are built into `docs/presentation/CE725_Team20_EVM_Dashboard.pptx` — 12 slides, 16:9, with the *Say* paragraphs carried through as speaker notes.

The deck is **generated**, not hand-built, by `docs/presentation/build_deck.js`:

```
NODE_PATH=$(npm root -g) node docs/presentation/build_deck.js
```

Every figure quoted on a slide is listed in one `FIG` block at the top of that script, so a number on a slide can be checked against the engine rather than taken on trust — the same reason the rest of the project is built the way it is. Slides 6 and 7 carry the dashboard screenshots from `docs/img/`, so they work unchanged if the live demonstration cannot be run.

---

1. Slide content and timing

Total 15 minutes. Timings are cumulative. The live demonstration is the centre of it — slides 6 to 9 are driven from the running dashboard, not from bullet points.

Slide 1 — Title (0:00–0:30)

Earned Value Management Dashboard Construction Economics and Finance · CE725 · Topic 7 Team 20 · Sajjad Nakhwa · Deependra Arya · Neel Shah · Akshat Karkar · Pearsh Gumasta

Say: "We built a tool that tells a project manager whether a project is in trouble, where the trouble is, and what it will cost. The part we want to spend most of the time on is how we know the numbers are right — and what that checking still cannot tell us."

Slide 2 — The problem, in one number (0:30–2:00)

> **A project has spent ₹3.98 crore against a plan of ₹4.02 crore. Is it healthy?**

Pause. Let the room answer "yes".

> It has completed ₹3.47 crore of work. It is **14.7% over budget** and **19 days behind programme**.

Say: "Two numbers give you one comparison. Budget against spend conflates 'am I doing the work' with 'is the work costing what it should'. Here the two errors point in opposite directions and cancel, and the report writes itself as healthy. You cannot separate them without a third measurement."

The third number: earned value — the budgeted cost of work actually performed.

Slide 3 — The method (2:00–3:30)

PV	Budgeted cost of work **scheduled**	What should have been done?
EV	Budgeted cost of work **performed**	What was done?
AC	Cost of doing it	What did it cost?

Everything else is a comparison of two of these three:

- EV vs AC → cost: **CV = EV − AC**, **CPI = EV / AC**
- EV vs PV → schedule: **SV = EV − PV**, **SPI = EV / PV**

Say: "That is the whole method. The rest is bookkeeping on top of it."

Slide 4 — The case study (3:30–4:30)

G+4 RCC academic block · BAC ₹14.00 crore · 12 months · 19 work packages · data date end of month 6

The story the numbers tell: - Hard rock in excavation — 45% over on that package, two months late - Cascaded into footings and foundation RCC - Frame started a month late, running an 18-day slab cycle against 12

Say, explicitly: "Synthetic data. Budgets are order-of-magnitude plausible for a building of this size, but they are assumed, not from any tender or invoice. We'll come back to why that choice matters."

Slide 5 — What we built (4:30–5:00)

Four pieces, one repository:

1. **Python engine** — all EVM measures, work-package level
2. **Streamlit dashboard** — the tool
3. **Live-formula Excel export** — for people who work in Excel
4. **Independent manual benchmark** — how we know it is right

→ **Switch to the live dashboard.**

Slide 6 — DEMO: the position (5:00–7:00)

Show the top of the dashboard.

Point at, in order: 1. The verdict bar — **BEHIND SCHEDULE AND OVERSPENT** 2. The plain-language summary — read it aloud; note it quotes no index without the consequence beside it 3. **CPI 0.872** — "every rupee spent returns 87 paise of budgeted work" 4. **TCPI 1.051** — "the remaining work must run at 1.051 to finish on budget. We have sustained 0.872 for six months. That is a 20.6% improvement on performance never once achieved. The budget is gone; what remains is deciding who absorbs it."

If you have time for only one sentence in the whole demonstration, it is the TCPI one.

Slide 7 — DEMO: where the variance is (7:00–9:00)

Open **Where the variance is**.

Say: "A project CPI of 0.872 says something is wrong. It does not say what to do. This does."

Point at: - Excavation, **CPI 0.689**, **100% complete** — "this variance is **final**. No recovery plan touches it." - Frame packages, **22% and 14% complete** — "these can still move." - "₹38 lakh of the ₹51.20 lakh is in completed substructure. Only ₹7.7 lakh is live. Stop trying to recover the substructure, protect the frame, and tell the client ₹38 lakh is gone."

This is the slide that justifies carrying the WBS through the whole calculation instead of rolling up to one project curve.

Slide 8 — DEMO: the schedule defect (9:00–10:30)

Open **Performance trend**.

Say: "SPI reads 0.863. There is a problem with that number, and it is structural.

SPI = EV / PV. At completion, EV must equal BAC and PV must equal BAC. **So SPI returns to 1.00 on every project ever completed, however late.** The index gets more reassuring as the project gets later.

Earned Schedule fixes it by asking *when* on the baseline this much value was scheduled to have been earned. That gives a time: **19 days behind**. A site manager can act on nineteen days. Nobody can act on 0.863."

Also point at **CPI flat at 0.86–0.88 across all six periods** while SPI swings to 0.74 and back: "cost performance stabilises early and rarely recovers. That is the strongest argument against an optimistic recovery plan."

Slide 9 — DEMO: the forecast (10:30–11:30)

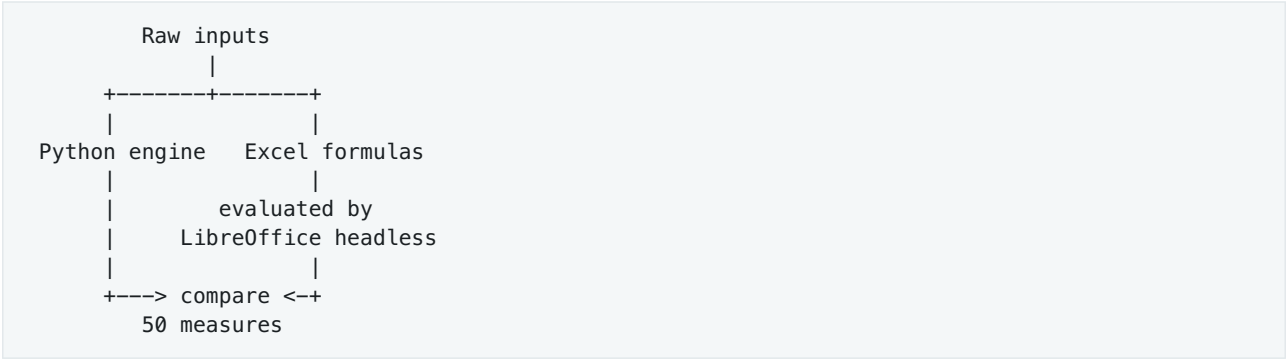
Open **Forecast**.

Four methods, ₹14.51 Cr to ₹17.98 Cr.

Say: "We deliberately do not pick one. They differ in a single assumption — what you believe about the remaining work. The ₹3.47 crore spread is the method being honest. Narrowing it needs a judgement nobody can make arithmetically: was the rock a closed event, or a symptom of a site that was never properly investigated?"

→ **Back to slides.**

Slide 10 — How we know it is right (11:30–13:00)



50 of 50 agree — ₹0.01 on currency, 1e−9 on indices, both periods. **151 automated tests.** Expected values hand-derived before the engine was written.

Say: "The critical decision was **not** to write a second Python implementation. A Python re-implementation validates only your own understanding — if you misread the formula, both sides misread it identically and the check passes. So the benchmark is in a different execution environment, and it is *executed*, not re-implemented."

Slide 11 — What we got wrong (13:00–14:00)

Six discrepancies, all recorded:

Found	Resolution
EAC3 off by **₹156**	**Our hand calc was wrong**, not the engine — rounded CPI×SPI mid-chain. Rule adopted: rounding is presentation only
Two validation tests failing	**The tests were wrong, not the engine.** Fixing the code would have deleted a correct validation rule
Executive summary **invisible** under dark theme	No test caught it. Every number right, tool unusable
Carry-forward routine fragile on gaps	Caught in review. Clean data hid it; real data would not

Say: "The second one matters most. The fast route to a green test suite was to change the engine until the tests passed, and that would have removed a rule that rejects earned value being double-counted."

Slide 12 — What validation cannot do (14:00–15:00)

The closing point. Do not rush it.

> Both implementations read **the same inputs**.
>
> If a package is reported **60% complete when it is really 40%**: the engine computes a wrong EV · the benchmark computes **the same** wrong EV · all 50 comparisons agree · every index is confidently wrong.

Earned value is percent complete × budget. Percent complete is a judgement made at site by someone who may be paid more if it is higher.

Say: "We validated the arithmetic to nine decimal places. The arithmetic was never the risk. And every design choice that makes this dashboard more persuasive makes the problem worse — it renders an unverifiable judgement with six-figure precision, a traffic light and a chart.

The quality of an EVM system is the quality of its percent-complete measurement. Everything else, including all of our work, is arithmetic on top of a judgement made by a person standing on a site."

---

2. Demonstration checklist

Before presenting:

- [ ] `.venv/bin/streamlit run app/streamlit_app.py` — started and warm
- [ ] Browser at 100% zoom, window maximised
- [ ] Reporting period slider at **6**
- [ ] Variance breakdown set to **Sub-level**
- [ ] A pre-built `output/EVM_Report.xlsx` open in a second window, in case the live export is slow
- [ ] `.venv/bin/python -m pytest tests/ -q` run once — be ready to show **151 passed** if asked
- [ ] Repository open in a browser tab

If the app fails to start, present from the screenshots in `docs/img/` and the exported workbook. Do not debug live.

---

3. Viva preparation

Any member may be asked anything. These are the questions we expect.

On the formulas

Q. Derive CPI and explain what 0.872 means physically. CPI = EV / AC = ₹3,47,30,000 / ₹3,98,50,000 = 0.872. Of every rupee spent, 87.2 paise bought work that was budgeted at that value. The other 12.8 paise bought nothing that appears in the budget.

Q. Why can SPI not be trusted near the end of a project? SPI = EV / PV. At completion EV = BAC and PV = BAC by definition, so SPI → 1.00 regardless of how late the project finished. The index improves as the project gets later. Earned Schedule avoids this by measuring in time.

Q. Compute Earned Schedule for period 6. Cumulative PV is ₹3,16,63,556 at P5 and ₹4,02,48,444 at P6. EV is ₹3,47,30,000, which lies between them. ES = 5 + (3,47,30,000 − 3,16,63,556) / (4,02,48,444 − 3,16,63,556) = 5 + 30,66,444 / 85,84,889 = **5.357 months**. SV(t) = 5.357 − 6 = −0.643 months ≈ 19 days behind.

Q. Why are there four EAC formulas? They differ in one assumption: the cost performance of the remaining work. EAC1 assumes it continues as now. EAC2 assumes the variance was a one-off. EAC3

assumes schedule pressure also continues, which normally costs money because acceleration does. EAC4 weights cost and schedule.

Q. Show that TCPI against EAC1 equals CPI. TCPI = (BAC − EV)/(EAC − AC). Substituting EAC = BAC/CPI = BAC·AC/EV: denominator = BAC·AC/EV − AC = AC(BAC − EV)/EV. So TCPI = (BAC − EV)·EV / (AC(BAC − EV)) = EV/AC = **CPI**. *Meaning:* a forecast built on "performance continues" demands, by construction, no change in performance.

Q. When is TCPI undefined? When AC ≥ BAC. The denominator (BAC − AC) is zero or negative: there is no budget left to do the remaining work with, at any efficiency. The tool returns NaN rather than a number.

Q. Why is CV negative but you call it 14.7%? CV% = CV / EV, expressed against the work done rather than the budget. −₹51,20,000 / ₹3,47,30,000 = −14.7%. The work completed cost 14.7% more than it was budgeted to cost.

On the data

Q. Where did these numbers come from? They are synthetic, generated by `data/generate_case_study.py`, and we say so in every document. Budgets are order-of-magnitude plausible for an institutional building at CPWD DSR-style rates, but they are assumed. Real work-package cost data is commercially sensitive and would have required a confidentiality agreement we could not obtain in four weeks.

Q. Isn't synthetic data a weakness? Yes, and we state it. We cannot claim the tool has been tested against missing periods, retrospective corrections or miscoded costs. What synthetic data bought is a case that exercises every branch of the method — a complete over-budget package whose variance is final, a part-complete one that can still move, a cascading slip.

Q. How is the baseline phased? Each package has a start, a finish and a curve shape. S-shaped packages use smoothstep, $3t^2 - 2t^3$, which has zero slope at both ends so a package ramps up and tails off. The project S-curve is the aggregate of 19 staggered packages — it emerges from the phasing rather than being imposed.

On the tool

Q. Why Streamlit and not Excel? Both. Streamlit for interrogation — the period slider, the WBS drill-down. Excel for distribution, because a site QS works in Excel and a report they cannot re-run is one they will retype. The export is live formulas, not pasted values.

Q. Why is the commentary not generated by a language model? Three reasons. Reproducibility: the same inputs must give the same sentence every month. Non-invention: a model given a table of indices will write a fluent paragraph containing a number that is not in the table. Auditability: every threshold is written down in one file and can be argued with. The tool is deliberately less AI-powered than it could be.

Q. What happens if I feed it bad data? It refuses to load rather than computing an index. EVM output is three ratios near 1.00 with no sanity check a reader can apply, so a CPI of 0.94 from corrupt data looks exactly like one from good data. Refusing is safer.

On validation

Q. How do you know the tool is right? Two independent implementations — Python and typed Excel formulas — agree on 50 measures across two periods, to ₹0.01 and 1e−9. The Excel side is evaluated by LibreOffice, not re-implemented in Python.

Q. Why not just write a second Python version? Because it would validate our understanding against itself. If we misread a formula, the second implementation embeds the same misreading and the check passes while both are wrong.

Q. What did validation find? Four things. The most instructive: EAC3 differed by ₹156 and **our hand calculation was wrong**, not the engine — we had rounded CPI × SPI mid-chain. Also two mis-constructed tests where the engine was right, and a CSS bug that made the executive summary invisible and that no test could have caught.

Q. What does validation NOT prove? That the inputs are right. Both implementations read the same percent-complete figures. A package reported at 60% when it is really 40% produces the same wrong EV on both sides, all 50 comparisons agree, and every index is confidently wrong. Only a measurement at site catches that.

On practice

Q. Would a contractor actually use this? A client's PMC most readily — they already receive summarised progress and want an independent recomputation. A contractor's planner would need schedule integration first: the tool reads a time-phased budget, not a network, so it cannot say whether a slip is on the critical path.

Q. What is the biggest barrier to EVM adoption? Not computation. EVM requires someone to state monthly, in writing, what fraction of each package is complete, and then be held to it when that figure meets the cost data. That is a transfer of accountability, and it is resisted for that reason. Better software does not fix it.

On AI use

Q. How much of this did AI write? Most of the code, and first drafts of the documents. All of it was checked: expected values hand-derived before the engine ran, an independent benchmark, and a 35-entry usage log recording where AI was wrong, where we were wrong, and where its output was rejected on principle.

Q. Give an example of AI being wrong. Its first case study spread the variance evenly across packages, which would have made the tool's central feature pointless. Its first validation report led with "50 of 50 agree" and buried the limitation that matters more. And it described a cost index as "behind plan" — schedule idiom applied to cost.

Q. Give an example of AI being right when you were wrong. The ₹156 EAC3 discrepancy. We assumed the engine was wrong. It recomputed in 40-digit decimal arithmetic and showed our hand figure had rounded mid-chain.

Q. Your charter calls this an "AI-assisted EVM dashboard", but you say the commentary does not use a language model. Where is the AI? In how it was built, not in what it runs. AI wrote most of the code and the first drafts of the documents, and that is logged in full. The delivered tool is deterministic by choice — no model is called at runtime, because a cost report must give the same sentence from the

same inputs every month and must not contain a figure that was never computed. Deciding *not* to put a language model in the output path, and being able to say why, is the substantive finding of the project rather than a shortfall against the charter.

Q. Did AI write your individual reflections? It drafted a scaffold — the four questions and a reminder of which recorded discrepancy each of us owned — and each of us then wrote our own entry. That is logged as entry 31. A reflection is a first-person claim about one person's understanding, so an AI-written one is worth nothing however well it reads, and any of us can be asked about our own paragraph.

Q. Who is accountable if a decision on these numbers goes wrong? The engineer who signs the report. Not the tool, not the AI. That is why every number is traceable to an input and a formula, and why the tool refuses to compute from inconsistent data.